

String Module

Namespace: [FSharp.Core](#)

Assembly: FSharp.Core.dll

Functional programming operators for string processing. Further string operations are available via the member functions on strings and other functionality in [System.String](#) and [System.Text.RegularExpressions](#) types.

Functions and values

Function or value	Description	
String.collect mapping str	▼ Builds a new string whose characters are the results of applying the function <code>mapping</code> to each of the characters of the input string and concatenating the resulting strings. <code>mapping</code> : char -> string The function to produce a string from each character of the input string. <code>str</code> : string The input string. Returns: string The concatenated string. Example The following samples shows how to interspace spaces in a text <pre>let input = "Stefan says: Hi!" input > String.collect (sprintf "%c ")</pre> The sample evaluates to "S t e f a n s a y s : H i ! " Example How to show the ASCII representation of a very secret text <pre>"Secret" > String.collect (fun chr -> int chr > sprintf "%d ")</pre> The sample evaluates to "83 101 99 114 101 116 "	
String.concat sep strings	▼ Returns a new string made by concatenating the given strings with separator <code>sep</code>, that is <code>a1 + sep + ... + sep + aN</code>. <code>sep</code> : string The separator string to be inserted between the strings of the input sequence. <code>strings</code> : string seq The sequence of strings to be concatenated. Returns: string A new string consisting of the concatenated strings separated by the separation string. ArgumentNullException Thrown when <code>strings</code> is null.	



Function or value

Description

Example

```

let input1 = ["Stefan"; "says: "; "Hello"; "there!"]

input1 |> String.concat " " // evaluates "Stefan says: Hello there!"

let input2 = [0..9] |> List.map string

input2 |> String.concat "" // evaluates "0123456789"
input2 |> String.concat ", " // evaluates "0, 1, 2, 3, 4, 5, 6, 7, 8, 9"

let input3 = ["No comma"]

input3 |> String.concat "," // evaluates "No comma"

```

String.exists predicate str

▼ Tests if any character of the string satisfies the given predicate.

predicate : [char](#) -> [bool](#)

The function to test each character of the string.

str : [string](#)

The input string.

Returns: [bool](#)

True if any character returns true for the predicate and false otherwise.

Example

Looking for uppercase characters

```

open System

"Yoda" |> String.exists Char.IsUpper // evaluates true

"nope" |> String.exists Char.IsUpper // evaluates false

```

String.filter predicate str

▼ Builds a new string containing only the characters of the input string for which the given predicate returns "true".

Returns an empty string if the input string is null

predicate : [char](#) -> [bool](#)

A function to test whether each character in the input sequence should be included in the output string.

str : [string](#)

The input string.

Returns: [string](#)

The resulting string.

Example

Filtering out just alphanumeric characters

```

open System

```



Function or value

Description

```
let input = "0 1 2 3 4 5 6 7 8 9 a A m M"

input |> String.filter Uri.IsHexDigit // evaluates "123456789aA"
```

Example

Filtering out just digits

```
open System

"hello" |> String.filter Char.IsDigit // evaluates ""
```

String.forall predicate str

▼ Tests if all characters in the string satisfy the given predicate.

predicate : [char](#) -> [bool](#)

The function to test each character of the string.

str : [string](#)

The input string.

Returns: [bool](#)

True if all characters return true for the predicate and false otherwise.

Example

Looking for lowercase characters

```
open System

"all are lower" |> String.forall Char.IsLower // evaluates false

"allarelower" |> String.forall Char.IsLower // evaluates true
```

String.init count initializer

▼ Builds a new string whose characters are the results of applying the function `initializer` to each index from `0` to `count-1` and concatenating the resulting strings.

count : [int](#)

The number of strings to initialize.

initializer : [int](#) -> [string](#)

The function to take an index and produce a string to be concatenated with the others.

Returns: [string](#)

The constructed string.

[ArgumentException](#)

Thrown when `count` is negative.

Example

Enumerate digits ASCII codes

```
String.init 10 (fun i -> int '0' + i |> sprintf "%d ")
```

The sample evaluates to: "48 49 50 51 52 53 54 55 56 57 "

Function or value	Description	≡
String.iter action str	▼ Applies the function <code>action</code> to each character in the string. <code>action</code> : char -> unit The function to be applied to each character of the string. <code>str</code> : string The input string. Example Printing the ASCII code for each character in the string <pre>let input = "Hello" input > String.iter (fun c -> printfn "%c %d" c (int c))</pre> The sample evaluates as <code>unit</code>, but prints: <pre>H 72 e 101 l 108 l 108 o 111</pre>	
String.iteri action str	▼ Applies the function <code>action</code> to the index of each character in the string and the character itself. <code>action</code> : int -> char -> unit The function to apply to each character and index of the string. <code>str</code> : string The input string. Example Numbering the characters and printing the associated ASCII code for each character in the input string <pre>let input = "Hello" input > String.iteri (fun i c -> printfn "%d. %c %d" (i + 1) c (int c))</pre> The sample evaluates as <code>unit</code>, but prints: <pre>1. H 72 2. e 101 3. l 108 4. l 108 5. o 111</pre>	
String.length str	▼ Returns the length of the string. <code>str</code> : string The input string. Returns: int The number of characters in the string.	



Function or value

Description

Example

Getting the length of different strings

```
String.length null    // evaluates 0
String.length ""      // evaluates 0
String.length "123"   // evaluates 3
```

[String.map mapping str](#)

▼ Builds a new string whose characters are the results of applying the function `mapping` to each of the characters of the input string.

mapping : [char](#) -> [char](#)

The function to apply to the characters of the string.

str : [string](#)

The input string.

Returns: [string](#)

The resulting string.

Example

Changing case to upper for all characters in the input string

```
open System

let input = "Hello there!"

input |> String.map Char.ToUpper // evaluates "HELLO THERE!"
```

[String.mapi mapping str](#)

▼ Builds a new string whose characters are the results of applying the function `mapping` to each character and index of the input string.

mapping : [int](#) -> [char](#) -> [char](#)

The function to apply to each character and index of the string.

str : [string](#)

The input string.

Returns: [string](#)

The resulting string.

Example

```
input |> String.mapi (fun i c -> (i, c))
```

Evaluates to [(0, '0'); (1, 'K'); (2, '!')].

[String.replicate count str](#)

▼ Returns a string by concatenating `count` instances of `str`.

count : [int](#)

The number of copies of the input string will be copied.

str : [string](#)

Function or value	Description	
	The input string.	
	Returns: <code>string</code>	
	The concatenated string.	
	ArgumentException	Thrown when <code>count</code> is negative.
Example		
	<code>"Do it!" > String.replicate 3</code>	
	Evaluates to <code>"Do it!Do it!Do it!"</code> .	